❓ Domande Modulo 2

✏️ Processing Images for Neural Networks

▼ Spiega perché rappresentare un'immagine come un tensore di forma (B, C, H, W) è più naturale per le reti neurali rispetto ad appiattirla immediatamente in un vettore.

Rappresentare un'immagine come un tensore di forma `(B, C, H, W)` (Batch, Canali, Altezza, Larghezza) è più naturale perché preserva la struttura spaziale dei dati. Le reti neurali, e specificamente i modelli convoluzionali, sono progettati appositamente per sfruttare questa struttura spaziale. Al contrario, sebbene trattare le immagini come vettori in $\mathbb{R}^n$ sia matematicamente conveniente nella formulazione classica dei problemi inversi, questa operazione di appiattimento (flattening) nasconde e fa perdere le informazioni strutturali spaziali essenziali per l'analisi convoluzionale.

▼ Nel modello di misurazione $\boldsymbol{y}^\delta = K\boldsymbol{x}^\dagger + \boldsymbol{e}$, discuti quali parti del problema potrebbero essere già influenzate da scelte di pre-elaborazione come la normalizzazione o il ritaglio (cropping).

Le scelte di pre-elaborazione, definite nel documento come "data preparation", incorporano implicitamente assunzioni sul problema di ricostruzione stesso.
Nello specifico, operazioni come la normalizzazione influenzano direttamente la scala del problema e, di conseguenza, il comportamento e la dinamica della loss function durante l'ottimizzazione. Inoltre, queste operazioni definiscono una specifica classe di soluzioni ammissibili e possono favorire o penalizzare la ricostruzione di determinate strutture all'interno delle immagini, come ad esempio il contrasto o la sparsità. Nel contesto del codice, il ritaglio (ad esempio tramite transforms.Resize per ottenere una shape fissa) definisce le dimensioni spaziali dell'operatore computazionale K.

▼ Confronta la **ricostruzione end-to-end** e la **ricostruzione ibrida** dal punto di vista della velocità, dell'interpretabilità e del controllo della coerenza dei dati.

- **Velocità:** La ricostruzione **end-to-end** (dove una rete è addestrata a mappare direttamente y^δ in $x^\dagger$) è estremamente rapida in fase di inferenza, richiedendo un solo passaggio in avanti (*forward pass*). I metodi **ibridi** (che alternano step di reti neurali a inversioni matematiche di K) sono più lenti perché calcolano iterazioni multiple
- **Interoperabilità:** L'approccio **end-to-end** è considerato una "scatola nera" a bassa interpretabilità, poiché la rete apprende una mappatura complessa in cui è difficile isolare il contributo della fisica da quello dell'apprendimento. I metodi **ibridi** sono altamente interpretabili poiché mantengono nettamente separato l'operatore fisico K dalla rete neurale (che funge solo da "regolarizzatore" avanzato)
- **Coerenza dei dati (Data Consistency):** I modelli **end-to-end** offrono scarso controllo matematico: non c'è garanzia che la ricostruzione prodotta, se re-inserita nel modello simulato, riproduca i dati misurati originari. I metodi **ibridi**, invece, integrano ad ogni iterazione un esplicito modulo di *data consistency* che vincola la soluzione x a rispettare fedelmente le osservazioni fisiche ($K\boldsymbol{x} \approx \boldsymbol{y}^\delta$)

▼ Supponi che un dataset contenga immagini in scala di grigi salvate come `uint8`. Descrivi una pipeline di pre-elaborazione matematicamente sensata prima che queste immagini vengano utilizzate per l'addestramento.

Il formato `uint8` codifica i pixel con numeri interi che vanno da 0 a 255. Le reti neurali si addestrano in modo ottimale con piccoli valori in virgola mobile per evitare l'esplosione o la scomparsa dei gradienti. Una pipeline sensata e robusta prevede:

1. **Casting:** Convertire i dati da interi a un formato a virgola mobile, come `float32`
2. **Normalizzazione Min-Max:** Portare i valori nel range [0, 1] dividendo l'intero tensore per 255.0
3. **Aggiunta delle dimensioni (Unsqueeze):** Le librerie come PyTorch richiedono il formato (B, C, H, W). Un'immagine in scala di grigi nativa (H, W) deve essere prima espansa a (1, H, W) per codificare il singolo canale cromatico, e poi impacchettata dal `DataLoader` che aggiungerà la prima dimensione per il Batch

✏️ PyTorch, Automatic Differentiation, and Training Mechanics

▼ **Crea un tensore di forma (4,1,32,32)** e spiega cosa significa ogni dimensione in una pipeline di imaging.

In una pipeline di imaging, la convenzione standard per la forma (shape) dei tensori è `(B, C, H, W)`. Quindi, un tensore di forma `(4, 1, 32, 32)` significa:

- **4 (B - Batch size):** È la dimensione del batch, ovvero ci sono 4 immagini indipendenti processate contemporaneamente in questo "pacchetto".
- **1 (C - Canali):** È il numero di canali. Un valore di 1 indica che le immagini sono in scala di grigi. Se fosse un'immagine a colori RGB o multispettrale, questo valore sarebbe 3 o superiore.
- **32 (H - Altezza):** L'altezza spaziale di ogni immagine è di 32 pixel.
- **32 (W - Larghezza):** La larghezza spaziale di ogni immagine è di 32 pixel.
  Questa forma strutturata è fondamentale perché gli operatori convoluzionali si aspettano esattamente questa disposizione per funzionare correttamente.

▼ **Scrivi un modello `Linear` minimale** e verifica manualmente un gradiente che PyTorch calcola automaticamente.

Il modello lineare è $y = Wx + b$, semplificato nell'esempio come $prediction = w + x$.
Codice PyTorch:

```
w = torch.tensor([2.0], requires_grad=True)
x = torch.tensor([3.0])
target = torch.tensor([1.0])
prediction = w + x  # Calcola 2.0 + 3.0 = 5.0
loss = torch.mean((prediction - target)  2)  # Calcola (5.0 - 1.0)^2 = 16.0
loss.backward()  # Calcola il gradiente
```

Verifica manuale:

$L(w) = (w + x - target)^2$

$\frac{\partial L}{\partial w} = 2 \cdot (w + x - target) \cdot 1$

Usando i valori del codice: $\frac{\partial L}{\partial w} = 2 \cdot (2.0 + 3.0 - 1.0) = 2 \cdot (4.0) = 8.0$

Il metodo loss.backward() automatizza l'applicazione della regola della catena (chain rule) calcolando questo valore esatto e salvandolo in w.grad.item().

▼ **Spiega perché l'ottimizzazione a *minibatch*** cambia il problema di ottimizzazione numerica anche quando la loss matematica è invariata. In particolare, spiega perché lo SGD a minibatch porta a una procedura di ottimizzazione stocastica.

L'ottimizzazione a minibatch cambia il problema perché non si calcola il gradiente sull'intero dataset (che darebbe un gradiente deterministico "vero"), ma solo su un piccolo sottoinsieme (batch) estratto ad ogni iterazione.

Ciò significa che il calcolo del gradiente $\nabla L(\Theta_k) \text{ in } \Theta_{k+1} = \Theta_k - \eta \nabla L(\Theta_k)$ è solo un'approssimazione del gradiente totale. Poiché i campioni nel batch cambiano (sono "mescolati" o "shuffled" ad ogni epoca dal DataLoader), la direzione di discesa calcolata varia casualmente attorno alla direzione ideale. Questo introduce un

elemento di casualità (stocasticità) nel processo di aggiornamento dei pesi, motivo per cui l'algoritmo viene definito **S**tochastic **G**radient **D**escent (SGD).

▼ **Fornisci un esempio** in cui una scelta implementativa di basso livello, come il *padding* o il `dtype`, cambia il modello effettivo che viene addestrato.

I dettagli implementativi possono influenzare il modello matematico più di quanto ci si possa aspettare. Un esempio fornito riguarda la scelta del **tipo di dato (dtype)** e la **precisione finita**:

Usare float32 (singola precisione) rispetto a float64 (doppia precisione) in problemi inversi mal condizionati. Poiché l'aritmetica a precisione finita accumula errori di arrotondamento attraverso molti livelli, e poiché nei problemi mal condizionati piccole perturbazioni dell'input vengono amplificate enormemente, la scelta di float32 rispetto a float64 può causare differenze visibili nell'immagine ricostruita finale. Il tipo di dato definisce fisicamente un compromesso tra uso della memoria e precisione algoritmica, cambiando la stabilità numerica del modello.

▼ **Nella demo di deblurring precedente**, identifica quale parte della pipeline è il modello *forward*, quale parte è la mappa inversa addestrabile e come i gradienti vengono propagati attraverso l'intero calcolo.

- **Modello Forward:** È l'operatore basato sulla fisica che simula la degradazione. Nel codice è identificato dall'operatore IPPy `K = operators.Blurring(...)`. La misurazione degradata si ottiene con `y_batch = K(x_batch)` (seguita dall'aggiunta di rumore).
- **Mappa Inversa Addestrabile:** È la rete neurale PyTorch `SmallDeblurCNN`, istanziata come `model`. Questa rete riceve in input i dati corrotti (`y_batch`) e tenta di invertire il processo producendo un'immagine pulita: `prediction = model(y_batch)`.
- **Propagazione dei Gradienti (Backpropagation):**
  1. Si calcola l'errore tra la predizione della rete e il dato reale pulito (`x_batch`) usando la loss MSE: `loss = loss_fn(prediction, x_batch)`.
  2. Si azzerano i gradienti precedenti con `optimizer.zero_grad()`.
  3. Si chiama `loss.backward()`. Questo comando innesca la differenziazione automatica di PyTorch che, registrando il grafo computazionale (operazioni tensoriali e livelli convoluzionali di `SmallDeblurCNN`), applica sistematicamente la chain rule all'indietro partendo dallo scalare `loss` per calcolare il gradiente rispetto a tutti i parametri Θ interni al modello.
  4. I parametri vengono infine aggiornati chiamando `optimizer.step()`.

✏️ From Machine Learning to Neural Networks

▼ Mostra esplicitamente che la composizione di tre mappe lineari è ancora lineare. Procedendo per ricorsione, dimostra che la composizione di un qualsiasi numero di modelli lineari è lineare.

Il documento definisce un modello lineare come $f_\Theta(x) = Wx + b$. Viene mostrata la composizione di due mappe lineari:

Siano $f_1(x) = W_1x + b_1$ e $f_2(z) = W_2z + b_2$.

La loro composizione è: $f_2(f_1(x)) = W_2(W_1x + b_1) + b_2 = W_2W_1x + W_2b_1 + b_2$.

Possiamo definire una nuova matrice $\tilde{W}_2 = W_2W_1$ e un nuovo vettore di bias $\tilde{b}_2 = W_2b_1 + b_2$, ottenendo una nuova trasformazione affine (lineare con bias) della forma $\tilde{W}_2x + \tilde{b}_2$.

Composizione di tre mappe lineari:

Sia ora $f_3(v) = W_3v + b_3$. Componiamola con il risultato precedente:

$f_3(f_2(f_1(x))) = W_3(W_2W_1x + W_2b_1 + b_2) + b_3$

$f_3(f_2(f_1(x))) = W_3W_2W_1x + W_3W_2b_1 + W_3b_2 + b_3$

Definendo $\tilde{W}_3 = W_3W_2W_1$ e $\tilde{b}_3 = W_3W_2b_1 + W_3b_2 + b_3$, otteniamo $\tilde{W}_3x + \tilde{b}_3$, che dimostra come la struttura sia ancora una singola mappa affine. Questo dimostra esplicitamente che la composizione di tre mappe è lineare.

Dimostrazione per ricorsione:

- **Caso base (n=1):** $f_1(x) = W_1x + b_1$ è lineare per definizione.
- **Ipotesi induttiva:** Assumiamo che la composizione di k modelli lineari produca una mappa affine (lineare) della forma $g_k(x) = \tilde{W}_kx + \tilde{b}_k$.
- **Passo induttivo (n=k+1):** Sia $f_{k+1}(z) = W_{k+1}z + b_{k+1}$ un nuovo modello lineare.

  Componendo f_{k+1} con $g_k(x)$, otteniamo: $f_{k+1}(g_k(x)) = W_{k+1}(\tilde{W}_kx + \tilde{b}_k) + b_{k+1} = (W_{k+1}\tilde{W}_k)x + (W_{k+1}\tilde{b}_k + b_{k+1})$.

  Poiché $(W_{k+1}\tilde{W}_k)$ è una matrice e $(W_{k+1}\tilde{b}_k + b_{k+1})$ è un vettore costante, l'espressione risultante ha ancora la forma $Wx + b$, ed è quindi una mappa affine (lineare). Questo conferma formalmente perché "la profondità conta, ma solo una volta che la linearità è stata interrotta" tramite funzioni di attivazione non lineari.

▼ Spiega con parole tue perché la funzione di loss cambia lo stimatore statistico appreso dalla rete.

La funzione di loss (perdita) è fondamentale perché guida il modo in cui i pesi della rete vengono aggiornati durante l'addestramento, determinando così la natura matematica della soluzione. Come spiegato nel testo, anche in un approccio end-to-end, una rete neurale si comporta implicitamente come uno stimatore statistico, in particolare come uno stimatore MAP (Maximum A Posteriori) per un problema variazionale implicito $\hat{x}(y^\delta) = \arg\min x(\mathcal{D}(Kx, y^\delta) + \mathcal{R}(x))$.

La funzione di loss che scegliamo di minimizzare durante il training $L(\Theta) = \sum l(f\Theta(x^{(i)}), y^{(i)})$ costringe la rete ad "assorbire" nei suoi pesi un certo modello implicito della fisica e della probabilità a priori delle immagini. Di conseguenza, se cambiamo la funzione di loss (es. usando un errore quadratico medio piuttosto che un'altra norma per valutare la coerenza dei dati), modifichiamo implicitamente i termini $\mathcal{D}$ e $\mathcal{R}$ matematici che la rete sta cercando di ottimizzare senza saperlo. Cambiare la loss significa quindi richiedere alla rete di bilanciare in modo diverso la coerenza ai dati con la "conoscenza a priori" appresa, cambiando di fatto il tipo di stimatore statistico che la rete implementa al termine dell'addestramento.

✏️ Convolutions, CNNs, and End-to-End Training

▼ Spiega perché il **padding** è necessario se si desidera che una rete convoluzionale produca un output con la stessa dimensione spaziale dell'input.

Una convoluzione è un'operazione in cui un kernel di dimensione $k \times k$ viene centrato e applicato su un'immagine di input. Quando il kernel viene applicato lungo i bordi dell'immagine, senza accorgimenti, "sporgerebbe" fuori dai confini originali. Per evitare errori, la convoluzione standard calcola il risultato solo dove il kernel si sovrappone completamente all'immagine, causando un restringimento (una riduzione delle dimensioni spaziali) nell'immagine di output.

Il **padding** (o convoluzione con padding) risolve questo problema circondando l'immagine di input con un numero adeguato di zeri prima di applicare il kernel. In pratica, per un kernel di dimensione $k \times k$, si aggiunge una cornice di zeri larga $\frac{k-1}{2}$ pixel su ciascun lato. In questo modo, l'operazione di convoluzione può essere applicata anche ai bordi originali dell'immagine, garantendo che l'output finale conservi esattamente la stessa dimensione spaziale dell'input. Questo è fondamentale per i compiti di ricostruzione end-to-end dove ci si aspetta che input e output abbiano la stessa forma.

▼ Spiega, con parole tue, perché le **CNN** sono più appropriate degli **MLP** per i compiti di ricostruzione delle immagini.

Le reti MLP (Multi-Layer Perceptron) sono inadeguate per le immagini perché elaborano ogni pixel in modo isolato, trattandoli come informazioni "puntuali". Tuttavia, nell'elaborazione delle immagini, il valore di un singolo pixel da solo dice molto poco: l'informazione cruciale per ricostruire un oggetto si trova nelle relazioni "locali" (un gruppo di pixel vicini) o "globali" (come i gruppi di pixel interagiscono nel contesto dell'immagine).

Le **CNN (Convolutional Neural Networks)** risolvono questo problema sostituendo i livelli lineari isolati con livelli convoluzionali. Le convoluzioni, per natura, operano su un "intorno" di pixel (definito dalla dimensione del kernel), estraendo informazioni locali e pattern spaziali (come bordi e texture). Inoltre, le CNN condividono gli stessi filtri (kernel) su tutta l'immagine, rendendo il modello *equivariante alla traslazione* (cioè capace di riconoscere un pattern indipendentemente da dove si trovi nell'immagine) e mantenendo il numero di parametri molto più contenuto ed efficiente rispetto a un MLP.

▼ Nell'impostazione dell'addestramento basata sull'operatore K, qual è il ruolo del termine di **rumore** e e perché è utile generarlo durante l'addestramento?

Nell'addestramento end-to-end sintetico, partiamo da un'immagine pulita x_i e generiamo la misurazione corrotta usando l'equazione $y_i^\delta = Kx_i + e$.

Il ruolo del termine e (come il rumore gaussiano) è quello di simulare la perturbazione reale che avviene nei sistemi di acquisizione.

È cruciale generare questo rumore (insieme alla sfocatura applicata da K) "al volo" durante l'addestramento per evitare il problema noto come **"inverse crime"**. Se si addestrasse la rete su dati puramente deterministici (senza rumore), la rete imparerebbe una mappatura perfetta in un ambiente "sterile", portando a stime di prestazioni irrealistiche quando testata su dati reali (che contengono sempre rumore ineliminabile). L'aggiunta del termine e forza la rete a imparare una ricostruzione robusta alle perturbazioni statistiche.

▼ Confronta le possibili **attivazioni finali** discusse in questo notebook. In quale situazione la **Tanh** risulta particolarmente naturale?

Le funzioni di attivazione finali discusse e confrontate per i task di elaborazione delle immagini sono:

- **Nessuna attivazione (lineare):** L'output può variare tra $-\infty$ e $+\infty$.
- **ReLU:** Ha il vantaggio logico di produrre solo valori non negativi, il che è ideale dato che i tensori d'immagine standardizzati variano tipicamente in $[0, 1]$. Questo aiuta specialmente a ricostruire sfondi perfettamente neri. Lo svantaggio è che non è differenziabile in 0, potendo causare instabilità durante l'addestramento in quella regione.
- **Sigmoide:** Il suo range è $(0, 1)$, teoricamente perfetto per un'immagine standardizzata. Tuttavia, è altamente instabile per l'ottimizzazione perché per raggiungere lo 0 esatto (il nero dello sfondo) o l'1, i pesi devono essere spinti verso l'infinito (poiché la sigmoide tende a 0 e 1 asintoticamente).
- **Tanh:** Ha un range compreso tra $(-1, 1)$. È instabile quanto la sigmoide se usata su CNN standard, ma **risulta particolarmente naturale e viene comunemente utilizzata quando si lavora con le CNN Residuali**.

👨‍🏫 Residual Learning, Receptive Field, and UNet

▼ Perché spesso è più facile apprendere il residuo piuttosto che l'immagine completa direttamente?

Nei problemi di elaborazione delle immagini (come il denoising o il deblurring), l'immagine di input corrotta e l'immagine target "pulita" sono solitamente molto simili. L'ottimizzatore trova molto più facile spingere i pesi di una rete verso zero (imparando così solo la piccola perturbazione o il "rumore" da sottrarre, ovvero il residuo) piuttosto che forzare una serie complessa di livelli non lineari ad approssimare una funzione identità per ricopiare l'intera struttura dell'immagine.

▼ Considera una CNN con tre livelli convoluzionali di dimensione del kernel 3 e stride 1. Qual è il suo campo ricettivo? Come cambia la risposta se un livello ha stride 2?

Utilizzando la formula del campo ricettivo: $RF_L = 1 + \sum_{l=1}^{L}(k_l - 1)\prod_{i=1}^{l-1} s_i$

- **Caso 1 (Tutti i livelli con stride 1):** $RF = 1 + (3-1)\cdot 1 + (3-1)\cdot 1 + (3-1)\cdot 1 = 1 + 2 + 2 + 2 = 7$.

  Il campo ricettivo è **7×7**.

- **Caso 2 (Se un livello ha stride 2):** Lo stride funge da moltiplicatore per l'espansione del campo ricettivo nei livelli successivi. Se, ad esempio, il primo livello avesse stride 2 ($s_1 = 2, s_2 = 1, s_3 = 1$), il calcolo diverrebbe: $RF = 1 + (3-1)\cdot 1 + (3-1)\cdot 2 + (3-1)\cdot 2 = 1 + 2 + 4 + 4 = 11$.

  Il campo ricettivo si espanderebbe a **11×11** (o 9×9 se lo stride 2 fosse nel secondo livello). Lo stride aumenta drasticamente "l'orizzonte visivo" della rete.

▼ Spiega perché il campo ricettivo è importante quando la corruzione è globale piuttosto che locale.

Se una corruzione è locale (es. rumore puntuale), alla rete basta guardare i pixel immediatamente vicini per correggerla. Ma se la corruzione è globale (come una forte sfocatura da movimento o artefatti a raggiera che attraversano l'intera immagine), un campo ricettivo piccolo non permette alla rete di comprendere la struttura anatomica sottostante; vedrebbe solo un pattern di artefatti confuso. Un campo ricettivo ampio è essenziale per aggregare il contesto spaziale da regioni distanti, permettendo al modello di risolvere le ambiguità e invertire la corruzione estesa in modo coerente.

▼ Qual è l'idea architettonica principale dietro la UNet, e perché le skip connections sono così importanti nei compiti di imaging?

La UNet si basa su un'architettura **Encoder-Decoder** . L'encoder estrae il contesto globale riducendo progressivamente la risoluzione spaziale (formando un collo di bottiglia), mentre il decoder ripristina la risoluzione originale.

Le **skip connections** (connessioni di salto) sono fondamentali perché l'operazione di contrazione dell'encoder distrugge le informazioni ad alta frequenza (dettagli fini, bordi esatti). Le skip connections aggirano il collo di bottiglia, passando le feature ad alta risoluzione direttamente dall'encoder al decoder. Questo permette alla rete di fondere il contesto globale astratto con la precisione spaziale locale.

▼ Confronta concettualmente una UNet standard e una Residual UNet. Quale vantaggio di ottimizzazione offrono i blocchi residui?

Una UNet standard utilizza sequenze di semplici livelli convoluzionali. Una Residual UNet (ResUNet) sostituisce questi livelli con **blocchi residui** in ogni fase (sia in codifica che in decodifica).

Il vantaggio principale di ottimizzazione è la mitigazione del problema del *vanishing gradient* (gradiente evanescente). Le scorciatoie dei blocchi residui permettono ai gradienti di fluire all'indietro in modo molto più pulito durante l'addestramento. Questo rende l'ottimizzazione più fluida e veloce, consentendo di addestrare varianti di UNet molto più profonde senza che le prestazioni si degradino.

▼ Qual è la motivazione dietro gli attention gates in una Attention UNet?

Nella UNet standard, le skip connections concatenano "alla cieca" tutte le feature dell'encoder con quelle del decoder, portando con sé non solo dettagli utili, ma anche rumore o background irrilevante. Gli **attention gates** agiscono come filtri morbidi: utilizzano le feature del decoder (che hanno già un forte significato semantico) come segnale guida per valutare le feature in arrivo dall'encoder. Questo meccanismo permette alla rete di "prestare attenzione" (evidenziare) solo alle regioni spaziali salienti ed eliminare quelle irrilevanti prima di fonderle.

▼ Con parole tue, quale problema sta cercando di risolvere la UNet++ con le skip connections nidificate?

La UNet++ mira a colmare il **"gap semantico"** tra l'encoder e il decoder. In una UNet normale, le feature di basso livello (encoder) vengono fuse direttamente con quelle di altissimo livello (decoder), il che può rendere difficile l'apprendimento per la rete. Le skip connections nidificate della UNet++ introducono percorsi intermedi e convoluzioni dense tra i due rami: in questo modo le feature dell'encoder vengono gradualmente "arricchite" semanticamente lungo il tragitto, arrivando al decoder molto più simili a ciò che il decoder si aspetta di ricevere.

Vision Transformers and Loss Design

▼ Perché un Vision Transformer ha bisogno di embedding posizionali una volta che l'immagine è stata divisa in patch?

A differenza delle reti convoluzionali (CNN), che elaborano le immagini facendo scorrere un kernel locale preservando implicitamente la griglia 2D, il meccanismo centrale dei Transformer (la *self-attention*) è un'operazione basata sugli insiemi ed è "invariante alle permutazioni" (*permutation-invariant*). Ciò significa che se non aggiungessimo alcun segnale, il modello mescolerebbe le patch senza sapere dove ciascuna si trovi fisicamente rispetto alle altre. Gli embedding posizionali sono vettori appresi o predefiniti che vengono sommati ai token delle patch per "reiniettare" l'informazione geometrica e spaziale nell'architettura, permettendo al modello di comprendere la struttura 2D originale dell'immagine.

▼ Spiega con parole tue perché la self-attention è interessante quando il compito di ricostruzione coinvolge interazioni a lungo raggio.

Le CNN classiche hanno un "campo ricettivo" limitato e locale: per capire la relazione tra due pixel molto lontani nell'immagine (ad esempio, un artefatto globale o una sfocatura molto ampia), la rete deve essere estremamente profonda affinché le informazioni possano gradualmente incrociarsi. La *self-attention*, al contrario, confronta direttamente *ogni* token (patch) con *tutti gli altri* token dell'immagine in un solo passaggio. Questo permette al modello di catturare immediatamente e in modo esplicito le dipendenze globali e a lungo raggio, risultando molto più efficace nella ricostruzione di difetti o geometrie che si estendono su tutta l'immagine.

▼ Qual è il principale compromesso (trade-off) indotto dalla dimensione della patch in un modello ViT?

Il compromesso principale riguarda **la risoluzione spaziale rispetto al costo computazionale**.

Se si riduce la dimensione della patch (es. da 16×16 a 8×8), il modello dividerà l'immagine in un numero molto maggiore di token, ottenendo una granularità e una risoluzione superiori per ricostruire i dettagli fini. Tuttavia, la complessità del calcolo della self-attention scala in modo **quadratico** $O(N^2)$ rispetto al numero totale di token (N). Di conseguenza, patch più piccole fanno esplodere il costo computazionale e l'uso di memoria, mentre patch più grandi rendono il modello veloce ma grossolano, rischiando di far perdere dettagli locali importanti.

▼ Confronta MSE e L1 come loss di addestramento. Che tipo di differenza ti aspetteresti nelle immagini ricostruite?

- La **MSE (Mean Squared Error o L2 Loss)** eleva al quadrato gli errori. Questo penalizza severamente i grandi scostamenti (outlier) ma tollera molto gli errori lievi. Dal punto di vista visivo, ciò spinge la rete a produrre una media sicura di tutte le possibili soluzioni, risultando spesso in immagini "sovrasfocate" (*over-smoothed*) dove i bordi netti e le texture fini vengono sbiaditi.
- La **L1 Loss (Mean Absolute Error)** penalizza l'errore in modo lineare. Risulta molto più robusta agli outlier e non cerca una media perfettamente smussata, incoraggiando piuttosto la rete a mantenere una maggiore fedeltà ai dettagli originali. Le ricostruzioni prodotte utilizzando L1 tendono a essere visibilmente più **nitide**, preservando meglio le alte frequenze e i bordi rispetto alla MSE.

▼ Perché si potrebbe voler combinare diverse loss invece di usarne solo una?

Perché ogni funzione di loss valuta l'immagine da una "prospettiva" diversa e presenta punti ciechi. Una loss pixel-wise (come L1 o MSE) garantisce che i colori e le intensità siano corretti localmente, ma non tiene conto della struttura globale percepita dall'occhio umano. Combinando le loss — ad esempio, accoppiando la L1 per l'accuratezza dei pixel, la SSIM (Structural Similarity Index) per preservare la struttura globale, e magari una loss percettiva (VGG) per le texture — si obbliga la rete a ottimizzare simultaneamente più aspetti qualitativi. Questo porta a ricostruzioni molto più bilanciate, realistiche e visivamente corrette.

Cross-Domain End-to-End Reconstruction

▼ Perché una mappatura convoluzionale diretta dal sinogramma all'immagine è di solito una scelta architettonica inadeguata?

Nei problemi inversi a dominio incrociato (come la Tomografia Computerizzata), l'input (sinogramma) e l'output (immagine anatomica) vivono in griglie spaziali con geometria e semantica completamente diverse. Le reti come le CNN o le U-Net sono costruite su convoluzioni **locali**, progettate per estrarre informazioni da un piccolo intorno di pixel. Tuttavia, un singolo punto fisico nell'immagine corrisponde a una curva sinusoidale **globale** che attraversa l'intero sinogramma. Una CNN non è in grado di mappare questa complessa dipendenza globale usando solo filtri locali, risultando in un forte disallineamento tra l'architettura della rete e la fisica del problema.

▼ Spiega perché ridimensionare un sinogramma alla dimensione dell'immagine non risolve il vero problema.

Ridimensionare (o fare il padding/cropping) del sinogramma per fargli avere la stessa forma (shape) $H \times W$ dell'immagine target risolve solo un problema informatico di dimensioni dei tensori, ma ignora la natura dei dati. Il disallineamento fondamentale è di natura **semantica**: i pixel del sinogramma rappresentano integrali di linea dei raggi X lungo angoli specifici, non l'intensità spaziale locale dell'anatomia. Anche se ridimensionato, i dati rimangono nel dominio delle proiezioni e una rete convoluzionale standard farà comunque un'enorme fatica a collegare quei pixel a quelli dell'immagine medica.

▼ Nella pipeline $y^\delta \mapsto \tilde{x} = \mathrm{FBP}(y^\delta) \mapsto f_\Theta(\tilde{x})$, qual è il ruolo del passaggio FBP (Filtered Back-Projection)?

Il passaggio FBP agisce come un **operatore di inversione del dominio**. Invece di forzare la rete neurale a imparare da zero la complessa trasformazione matematica dal dominio dei dati (sinogramma) a quello spaziale (immagine), si utilizza un algoritmo classico (la FBP) per sollevare i dati nello spazio corretto. La FBP produce una prima ricostruzione dell'immagine che, sebbene grezza e piena di artefatti (soprattutto se gli angoli di acquisizione sono pochi), si trova ora nello stesso dominio spaziale e semantico del target desiderato.

▼ Perché una UNet può operare efficacemente sulla ricostruzione FBP anche se avrebbe difficoltà sul sinogramma grezzo?

Una volta che la FBP ha proiettato i dati nel dominio dell'immagine, il problema si trasforma in un classico compito **image-to-image** (es. denoising o rimozione di artefatti). Gli artefatti introdotti dalla FBP (come rumore o striature a raggiera) si presentano ora come pattern spaziali sovrapposti all'anatomia. Le architetture come la U-Net, grazie alla loro struttura multiscala e alle convoluzioni locali, eccellono nell'identificare e rimuovere pattern spaziali indesiderati. La rete non deve più imparare la fisica della trasformata di Radon, ma deve solo "pulire" un'immagine, compito per il quale è ottimizzata.

👨‍🎨 Deep Generative Models, VAEs, and GANs

▼ Spiega la differenza tra un modello generativo esplicito e un modello generativo implicito.

- In un **modello generativo esplicito** (come i VAE), si scrive direttamente un modello probabilistico e si cerca di ottimizzare o approssimare una verosimiglianza (likelihood) calcolabile.
- In un **modello generativo implicito** (come le GAN), si definisce un meccanismo di campionamento (ad esempio $x = G_\Theta(z)$) senza avere necessariamente a disposizione una formula trattabile analiticamente per la distribuzione di probabilità $p_\Theta(x)$.

▼ In un modello a variabile latente profondo, qual è il ruolo della variabile latente z?

La variabile latente z agisce come una descrizione interna compressa dell'immagine. Invece di memorizzare ogni singolo pixel, essa codifica in modo compatto i fattori principali e le proprietà strutturali che spiegano l'immagine (ad esempio l'anatomia globale, le forme principali e le texture in ambito medico). L'uso di questa variabile suggerisce che le immagini complesse vivono in prossimità di una varietà (manifold) di dimensione molto inferiore all'interno dello spazio ad alta dimensionalità dei pixel.

▼ Deriva i due termini che compaiono nell'ELBO del VAE e spiegane il significato.

L'ELBO (Evidence Lower Bound) che il VAE cerca di massimizzare è definito come: $\mathbb{E}_{q_\phi(z|x)}[log\ p_\Theta(x|z)] - KL(q_\phi(z|x)||p(z))$.

- **Termine di ricostruzione ($\mathbb{E}[log\ p_\Theta(x|z)]$):** Forza il decodificatore a preservare informazioni sufficienti dal codice latente per poter ricostruire fedelmente l'immagine originale.
- **Termine di divergenza KL ($KL(q_\phi(z|x)||p(z))$):** Funge da regolarizzatore per la rappresentazione latente, costringendo la distribuzione a posteriori approssimata a rimanere vicina alla distribuzione a priori semplice (solitamente una Gaussiana standard).

▼ Perché il trucco della riparametrizzazione è importante per l'addestramento del VAE?

Il trucco della riparametrizzazione consiste nello scrivere il campionamento non in modo diretto, ma come $z = \mu + \sigma \odot \epsilon$, dove $\epsilon \sim \mathcal{N}(0, I)$. Questo è fondamentale perché mantiene esplicita la casualità del campionamento preservando al tempo stesso la differenziabilità dell'operazione rispetto ai parametri della rete, permettendo così il normale flusso della backpropagation durante l'addestramento.

▼ Qual è il ruolo del discriminatore o critico in una GAN?

Il discriminatore (o critico) è una rete neurale (D_Ψ) che riceve in input un'immagine e produce un punteggio che indica se l'immagine sembra reale o falsa (generata). Il suo ruolo è quello di competere in un gioco avversario contro il generatore, cercando di distinguere i campioni reali estratti dal dataset da quelli sintetici creati dal generatore.

▼ Perché WGAN e WGAN-GP sono spesso preferite rispetto all'obiettivo GAN originale nella pratica?

Le GAN originali sono notoriamente delicate e instabili da addestrare: il discriminatore può diventare troppo forte o troppo debole e il generatore può soffrire di *mode collapse* (imparando a produrre solo un piccolo sottoinsieme dei possibili output). Le varianti basate su Wasserstein (come WGAN e WGAN-GP) sono spesso preferite perché utilizzano un critico che emette uno scalare non vincolato e applicano controlli di stabilità mirati (come la limitazione di Lipschitz o la normalizzazione spettrale), rendendo l'ottimizzazione avversaria molto più robusta e fluida, specialmente nei problemi inversi.

👨‍🏫 Diffusion Models: DDPM and DDIM

▼ Qual è il ruolo del processo di diffusione in avanti in un DDPM?

Il processo di diffusione in avanti definisce una sequenza in cui un'immagine pulita viene progressivamente corrotta aggiungendo rumore gaussiano passo dopo passo, fino a diventare puro rumore indistinguibile. Questo processo distrugge sistematicamente l'informazione per fornire alla rete neurale i dati di addestramento ideali, affinché impari a invertire tale corruzione.

▼ Perché la formula per $q(x_t \mid x_0)$ è così importante nella pratica?

Questa formula è fondamentale perché permette di campionare lo stato rumoroso x_t a qualsiasi timestep direttamente dall'immagine originale x_0 in un singolo passaggio matematico. Sfruttando le proprietà delle gaussiane, si evita di dover simulare iterativamente tutti i passi di rumore intermedi, rendendo la generazione dei batch di addestramento per la rete estremamente veloce ed efficiente.

▼ Nell'addestramento DDPM, cosa viene richiesto di prevedere alla rete neurale?

L'obiettivo della rete neurale $\epsilon_\theta(x_t, t)$ è molto semplice: le viene richiesto di prevedere esclusivamente la componente di rumore ϵ che è stata aggiunta all'immagine originale x_0 per ottenere l'immagine rumorosa x_t in quello specifico istante temporale t.

▼ Spiega come si può stimare x_0 da x_t e dal rumore previsto.

Una volta che la rete ha previsto il rumore ϵ_θ , è possibile riorganizzare algebricamente l'equazione del processo in avanti per isolare la stima dell'immagine originale $\hat{x}_0$. In pratica, si sottrae da x_t il rumore previsto (moltiplicato

per il suo fattore di scala ($\sqrt{1-\bar{\alpha}_t}$) e si divide il risultato per il fattore di mantenimento del segnale $\sqrt{\bar{\alpha}_t}$.

▼ Perché il campionamento DDPM è solitamente più lento rispetto alla generazione con un VAE o una GAN?

Le reti come i VAE o le GAN generano un'immagine completa in un singolo "passaggio in avanti" (*forward pass*). Al contrario, per generare una nuova immagine con un DDPM si parte da puro rumore e si deve applicare iterativamente la rete neurale per centinaia o migliaia di passi (da T a 0) per rimuovere gradualmente il rumore, rendendo il processo computazionalmente molto più lungo.

▼ Qual è il principale vantaggio pratico del DDIM rispetto al DDPM?

Il vantaggio principale dei modelli DDIM è che permettono di accelerare massicciamente la generazione "saltando" interi step durante il processo inverso. Invece di dover valutare la rete per tutti i T passi previsti originariamente, i DDIM generalizzano il campionamento e riescono a produrre immagini di alta qualità valutando la rete solo in 10-50 passi.

🤖 Diffusion Models for Inverse Problems

▼ Nella formulazione bayesiana, quali sono i rispettivi ruoli della verosimiglianza (likelihood) e del prior?

Nella formulazione bayesiana, la ricostruzione combina due pezzi di informazione fondamentali.

- La **verosimiglianza (likelihood)** misura l'accordo o la coerenza della soluzione con i dati misurati.
- Il **prior** misura la plausibilità dell'immagine stessa, incorporando una conoscenza a priori su come dovrebbe apparire un'immagine realistica.

▼ Perché un modello di diffusione può essere interpretato come un prior d'immagine appreso?

I modelli di diffusione imparano a rimuovere il rumore da immagini realistiche attraverso molti livelli di rumore, fornendo un predittore di rumore e una stima dell'immagine pulita. Pur non fornendo una densità analitica calcolabile in forma chiusa, restituiscono informazioni di denoising, ovvero informazioni sullo "score" (punteggio) per versioni rumorose della distribuzione dei dati. Questo permette di guidare la ricostruzione iterativa indicando in quale direzione l'immagine diventa più realistica.

▼ Qual è la principale idea algoritmica alla base del DPS?

L'idea principale del DPS (Diffusion Posterior Sampling) è utilizzare un gradiente di verosimiglianza (likelihood gradient) durante il processo di diffusione inversa. Piuttosto che applicare una proiezione separata, il DPS utilizza la discrepanza rispetto alle misurazioni come un segnale di gradiente che modifica direttamente la traiettoria della diffusione inversa. Per fare ciò, utilizza la stima dell'immagine pulita ($\hat{x}_0$) prodotta dal denoiser e differenzia il termine di fedeltà ai dati attraverso tale stima.

▼ Qual è la filosofia plug-and-play alla base di DiffPIR?

La filosofia plug-and-play di DiffPIR consiste nell'alternare due operazioni distinte: un passaggio di denoising del modello di diffusione (che spinge l'iterata verso il prior d'immagine appreso) e un passaggio di coerenza dei dati (che la spinge nuovamente verso l'accordo con le misurazioni). Invece di definire una transizione posteriore esatta, DiffPIR tratta l'azione di denoising come un prior implicito e la combina modularmente con una correzione di discesa del gradiente guidata dall'operatore in avanti.